

Módulo: Visões (Views) em T-SQL (SQL Server)

Curso: Banco de Dados Relacionais aplicado à Administração Tributária **Público-**

alvo: Profissionais da Secretaria de Estado da Fazenda da Paraíba (SEFAZ-PB)

SGBD utilizado: Microsoft SQL Server (T-SQL) — ferramenta: SQL Server

Management Studio (SSMS) **Carga horária sugerida:** 4 horas (2h teoria + 2h prática)

Pré-requisitos: domínio de **SELECT**, **JOIN**, **GROUP BY/HAVING**, subconsultas e CTEs
(ver [subconsultas.md](#))

Sumário

1. [Objetivos do Módulo](#)
 2. [O que é uma Visão](#)
 3. [Criando e Removendo Visões](#)
 4. [Consultando Visões](#)
 5. [Visões e Segurança de Dados](#)
 6. [Visões Atualizáveis e Restrições](#)
 7. **WITH CHECK OPTION**
 8. **WITH SCHEMABINDING**
 9. [Visões Indexadas \(Materialized Views\)](#)
 10. **INFORMATION_SCHEMA** e Metadados de Visões
 11. [Visões x CTEs x Tabelas Temporárias: Quando Usar Cada Uma](#)
 12. [Desempenho e Plano de Execução](#)
 13. [Cenário Integrado: Painel de Auditoria Fiscal](#)
 14. [Boas Práticas](#)
 15. [Glossário](#)
 16. [Exercícios](#)
 17. [Referências](#)
-

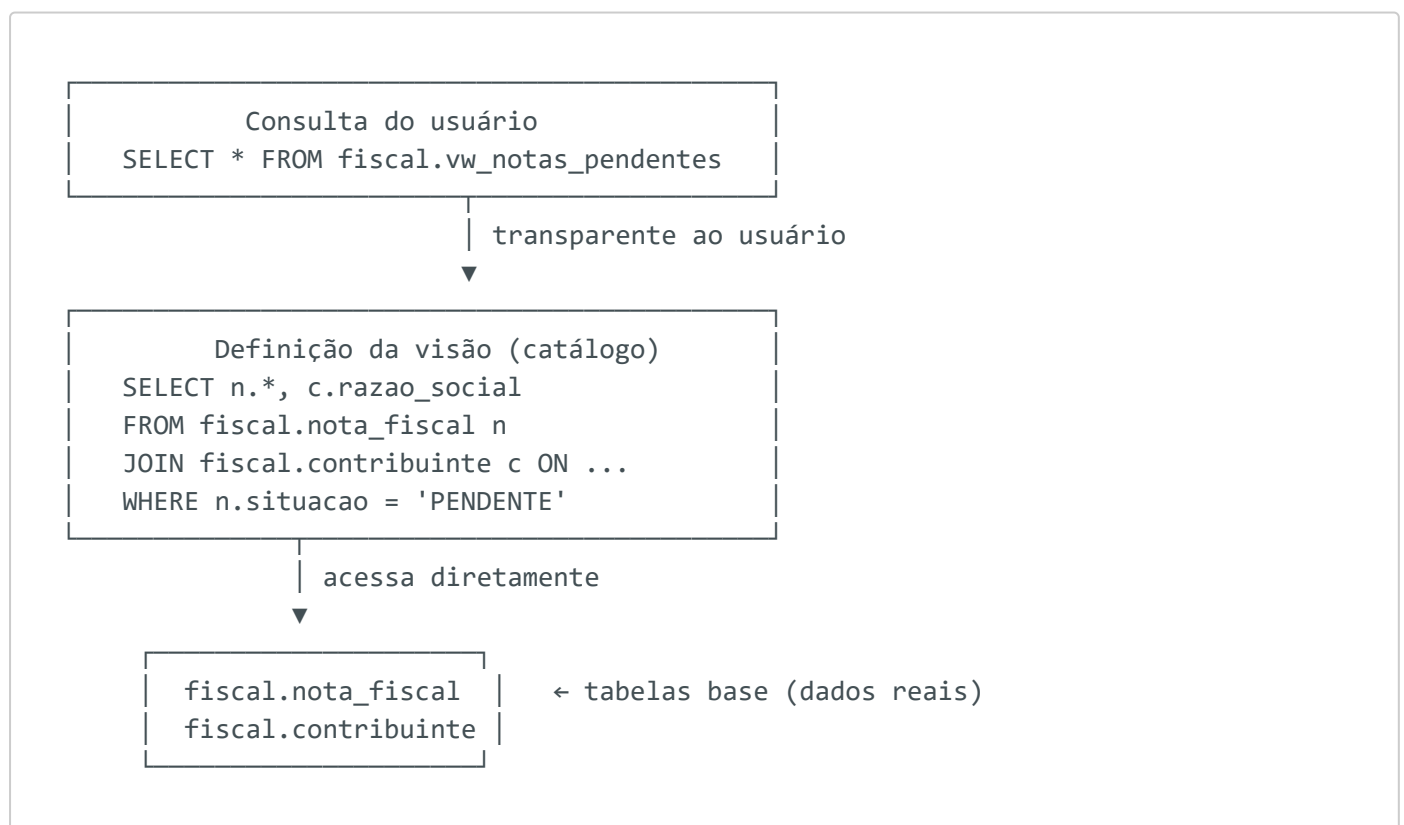
1. Objetivos do Módulo

Ao final deste módulo, o participante será capaz de:

- Definir o que é uma visão (*view*) e explicar a diferença entre visão e tabela base.
- Criar, modificar e remover visões com **CREATE VIEW**, **ALTER VIEW** e **DROP VIEW**.
- Utilizar visões para **simplificar consultas complexas** recorrentes no ambiente fiscal.
- Aplicar visões como mecanismo de **controle de acesso** — restringindo quais colunas e linhas cada perfil de usuário pode enxergar.
- Compreender as restrições de **atualização** em visões e usar **WITH CHECK OPTION** para garantir integridade.
- Usar **WITH SCHEMABINDING** para proteger a estrutura das tabelas base.
- Criar e consultar **visões indexadas** (*indexed views*) para acelerar relatórios fiscais agregados.
- Comparar visões com CTEs e tabelas temporárias, escolhendo a abordagem adequada para cada cenário.

2. O que é uma Visão

Uma **visão** (*view*) é um objeto de banco de dados que armazena uma instrução **SELECT** com um nome — funcionando como uma **tabela virtual**: não armazena dados próprios, mas apresenta o resultado da consulta definidora toda vez que é referenciada.



Vantagens no contexto da SEFAZ-PB:

Benefício	Como se aplica na auditoria fiscal
Simplificação	Audidores consultam <code>vw_notas_com_divergencia</code> sem precisar reescrever o <code>JOIN</code> de 5 tabelas toda vez
Segurança	O perfil "consulta pública" enxerga apenas colunas e linhas autorizadas, sem acesso às tabelas base
Padronização	A regra de negócio ("nota cancelada = situacao 'C'") fica centralizada na visão — muda-se em um lugar, vale para todos
Manutenção	Alterações no modelo físico das tabelas podem ser absorvidas na visão sem impactar aplicações clientes

O que uma visão NÃO é:

- Não é uma cópia dos dados — consultar a visão sempre lê as tabelas base em tempo real (exceto visões indexadas, ver [seção 9](#)).
- Não tem `ORDER BY` garantido — `ORDER BY` numa visão não garante a ordem de saída; quem define a ordem é o `ORDER BY` da consulta **externa** que usa a visão.
- Não é uma tabela temporária — visões são objetos permanentes do banco de dados, listados em `sys.views`.

3. Criando e Removendo Visões

3.1 `CREATE VIEW` — sintaxe básica

```
CREATE VIEW fiscal.vw_notas_emitidas_ativas
AS
SELECT
    n.nrchaveacesso      AS chave_acesso,
    n.dhmissao          AS data_emissao,
    n.vltotalnota       AS valor_total,
    n.stnfe             AS situacao,
    c.razao_social,
    c.cnpj,
    c.regime_tributario
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
    ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C'; -- exclui notas canceladas
```

Convenção de nomenclatura adotada neste curso: prefixo **vw_** para visões, seguido de nome descritivo em minúsculas com underscores. Ex.:

vw_notas_emitidas_ativas, vw_divergencias_icms.

3.2 CREATE OR ALTER VIEW — idempotente (SQL Server 2016+)

```
CREATE OR ALTER VIEW fiscal.vw_notas_emitidas_ativas
AS
SELECT
    n.nrchaveacesso      AS chave_acesso,
    n.dhmissao          AS data_emissao,
    n.vltotalnota       AS valor_total,
    n.stnfe             AS situacao,
    c.razao_social,
    c.cnpj,
    c.regime_tributario
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
    ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C';
```

CREATE OR ALTER VIEW cria a visão se não existir ou a substitui se já existir — eliminando o padrão **DROP VIEW IF EXISTS + CREATE VIEW** e evitando que permissões concedidas sobre a visão sejam perdidas durante a atualização.

3.3 ALTER VIEW — modificar uma visão existente

```

ALTER VIEW fiscal.vw_notas_emitidas_ativas
AS
SELECT
    n.nrchaveacesso      AS chave_acesso,
    n.dhemissao          AS data_emissao,
    n.vltotalnota        AS valor_total,
    n.vlicms              AS valor_icms,    -- coluna adicionada
    n.stnfe              AS situacao,
    c.razao_social,
    c.cnpj,
    c.regime_tributario
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
    ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C';

```

Importante: `ALTER VIEW` **preserva** as permissões já concedidas sobre a visão, ao contrário de `DROP + CREATE`.

3.4 DROP VIEW — remover uma visão

```

-- Remover uma visão
DROP VIEW IF EXISTS fiscal.vw_notas_emitidas_ativas;

-- Remover múltiplas visões em um único comando
DROP VIEW IF EXISTS
    fiscal.vw_notas_emitidas_ativas,
    fiscal.vw_divergencias_icms,
    fiscal.vw_contribuintes_omissos;

```

`DROP VIEW` remove apenas o objeto de visão — **jamais** apaga dados das tabelas base.

3.5 Restrições sintáticas do T-SQL para visões

Regra	Detalhe
<code>ORDER BY</code> proibido	Não é permitido <code>ORDER BY</code> na definição da visão, exceto quando usada com <code>TOP</code> ou <code>OFFSET...FETCH</code>
<code>SELECT *</code> não recomendado	Se uma coluna for adicionada à tabela base, ela não aparece automaticamente na visão — o T-SQL "congela"

Regra	Detalhe
	as colunas no momento da criação
Referências a objetos temporários proibidas	Visões não podem referenciar tabelas <code>#temp</code>
Múltiplas instruções proibidas	A definição da visão deve conter exatamente uma instrução <code>SELECT</code>
Não pode ter parâmetros	Diferente de funções, visões não aceitam parâmetros — para isso use funções com valor de tabela (TVF)

4. Consultando Visões

Uma vez criada, a visão é utilizada exatamente como uma tabela — em `SELECT`, `JOIN`, subconsultas, CTEs, etc.

4.1 Consulta simples sobre a visão

```
-- O auditor consulta como se fosse uma tabela, sem ver o JOIN interno
SELECT chave_acesso, data_emissao, valor_total, razao_social
FROM fiscal.vw_notas_emitidas_ativas
WHERE valor_total > 50000.00
ORDER BY valor_total DESC;
```

4.2 Filtrando e agregando sobre a visão

```
-- Total emitido por regime tributário no mês atual
SELECT
    regime_tributario,
    COUNT(*)           AS qtd_notas,
    SUM(valor_total)   AS total_emitido,
    AVG(valor_total)   AS ticket_medio
FROM fiscal.vw_notas_emitidas_ativas
WHERE MONTH(data_emissao) = MONTH(GETDATE())
      AND YEAR(data_emissao) = YEAR(GETDATE())
GROUP BY regime_tributario
ORDER BY total_emitido DESC;
```

4.3 Juntando a visão com outra tabela

```
-- Notas ativas da visão cruzadas com autos de infração abertos
SELECT
    v.chave_acesso,
    v.razao_social,
    v.valor_total,
    ai.numero_auto,
    ai.data_lavratura
FROM fiscal.vw_notas_emitidas_ativas AS v
JOIN fiscal.auto_infracao AS ai
    ON ai.chave_acesso_nf = v.chave_acesso
WHERE ai.situacao = 'ABERTO'
ORDER BY ai.data_lavratura DESC;
```

4.4 Visão usada em subconsulta

```
-- Contribuintes com média de nota acima de R$ 100.000 (usando a visão como tabela
derivada)
SELECT cnpj, razao_social, media_notas
FROM (
    SELECT
        cnpj,
        razao_social,
        AVG(valor_total) AS media_notas
    FROM fiscal.vw_notas_emitidas_ativas
    GROUP BY cnpj, razao_social
) AS resumo
WHERE media_notas > 100000.00
ORDER BY media_notas DESC;
```

5. Visões e Segurança de Dados

Este é um dos usos mais estratégicos de visões no ambiente fiscal. Através de visões, é possível implementar **controle de acesso em nível de coluna e de linha** sem precisar de lógica na aplicação.

5.1 Segurança em nível de coluna

O auditor externo não deve enxergar o CNPJ completo dos contribuintes (dado sensível sujeito à LGPD). Criamos uma visão que mascara o CNPJ:

```
CREATE VIEW fiscal.vw_notas_para_auditoria_externa
AS
SELECT
    n.nrchaveacesso                AS chave_acesso,
    n.dhemissao                    AS data_emissao,
    n.vltotalnota                  AS valor_total,
    n.vlicms                       AS valor_icms,
    n.stnfe                        AS situacao,
    -- CNPJ mascarado: exibe apenas os 8 primeiros dígitos (raiz do CNPJ)
    LEFT(c.cnpj, 8) + '*****'    AS cnpj_raiz,
    c.razao_social,
    c.regime_tributario,
    c.municipio
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
    ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C';
```

Em seguida, concedemos acesso **somente à visão**, jamais às tabelas base:

```
-- Concede permissão de SELECT na visão para o perfil de auditoria externa
GRANT SELECT ON fiscal.vw_notas_para_auditoria_externa TO
[perfil_auditoria_externa];

-- Nega acesso direto às tabelas base (se necessário explicitar)
DENY SELECT ON fiscal.nota_fiscal TO [perfil_auditoria_externa];
DENY SELECT ON fiscal.contribuinte TO [perfil_auditoria_externa];
```

5.2 Segurança em nível de linha (Row-Level Security via visão)

Cada analista fiscal deve enxergar apenas os contribuintes da sua circunscrição regional:

```
CREATE VIEW fiscal.vw_notas_minha_circunscricao
AS
SELECT
    n.nrchaveacesso AS chave_acesso,
    n.dhemissao     AS data_emissao,
    n.vltotalnota   AS valor_total,
    n.stnfe         AS situacao,
    c.razao_social,
    c.cnpj,
```



```

        c.municipio,
        c.id_circunscricao
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
    ON c.id_contribuinte = n.id_contribuinte_emitente
-- SUSER_SNAME() retorna o login do usuário atual – correlação dinâmica por login
WHERE c.id_circunscricao = (
    SELECT id_circunscricao
    FROM fiscal.analista_fiscal
    WHERE login_windows = SUSER_SNAME()
);

```

Nota de segurança: a função `SUSER_SNAME()` é avaliada em tempo de execução para cada consulta — não é possível burlar o filtro consultando a visão, pois o SQL Server resolve o login antes de executar o plano. Para ambientes com muitos usuários, considere o recurso nativo **Row-Level Security (RLS)** do SQL Server 2016+, que implementa esse padrão de forma mais robusta e centralizada.

5.3 Concedendo e revogando permissões

```

-- Concede acesso somente leitura à visão operacional
GRANT SELECT ON fiscal.vw_notas_emitidas_ativas TO [perfil_fiscal_operacional];

-- Permite que analistas sênior criem outras visões no schema
GRANT CREATE VIEW TO [perfil_analista_senior];
GRANT ALTER ON SCHEMA::fiscal TO [perfil_analista_senior];

-- Revoga acesso concedido anteriormente
REVOKE SELECT ON fiscal.vw_notas_emitidas_ativas FROM [perfil_fiscal_operacional];

```

6. Visões Atualizáveis e Restrições

Embora visões sejam conceitualmente tabelas virtuais, **nem toda visão aceita operações de escrita** (`INSERT`, `UPDATE`, `DELETE`). O T-SQL permite modificar dados através de uma visão desde que sejam respeitadas determinadas regras.

6.1 Regras para visões atualizáveis

Uma visão é atualizável quando:

Condição	Detalhe
A visão referencia apenas uma tabela base (sem JOIN)	Cada modificação pode ser mapeada de volta a uma única tabela
Não contém DISTINCT , GROUP BY , HAVING	Esses operadores tornam a saída não mapeável para linhas individuais
Não contém agregações (SUM , AVG , etc.)	Por mesma razão
Não contém UNION/INTERSECT/EXCEPT	O mapeamento de volta a tabelas base seria ambíguo
As colunas da tabela base com NOT NULL sem DEFAULT estão incluídas na visão	Para que um INSERT via visão possa preencher todas as colunas obrigatórias

6.2 Exemplo — visão atualizável (tabela única)

```
CREATE VIEW fiscal.vw_contribuintes_simples
AS
SELECT
    id_contribuinte,
    razao_social,
    cnpj,
    regime_tributario,
    municipio,
    situacao_cadastral
FROM fiscal.contribuinte
WHERE regime_tributario = 'Simples Nacional';
```

Esta visão **é atualizável** — referencia apenas **fiscal.contribuinte**:

```
-- UPDATE via visão: atualiza a tabela base fiscal.contribuinte
UPDATE fiscal.vw_contribuintes_simples
SET situacao_cadastral = 'IRREGULAR'
WHERE cnpj = '12345678000195';

-- INSERT via visão: insere na tabela base fiscal.contribuinte
INSERT INTO fiscal.vw_contribuintes_simples
    (razao_social, cnpj, regime_tributario, municipio, situacao_cadastral)
VALUES
    ('Padaria Bom Pão Ltda', '98765432000111', 'Simples Nacional', 'João Pessoa',
    'REGULAR');
```

6.3 Exemplo — visão NÃO atualizável (JOIN, agregação)

```
-- Esta visão NÃO aceita UPDATE/INSERT/DELETE diretamente
CREATE VIEW fiscal.vw_resumo_por_regime
AS
SELECT
    c.regime_tributario,
    COUNT(*) AS qtd_contribuintes,
    SUM(n.vltotalnota) AS total_notas
FROM fiscal.contribuinte AS c
JOIN fiscal.nota_fiscal AS n
    ON n.id_contribuinte_emitente = c.id_contribuinte
GROUP BY c.regime_tributario;
```

Tentar `UPDATE fiscal.vw_resumo_por_regime SET ...` resultará no erro:

```
View or function 'fiscal.vw_resumo_por_regime' is not updatable because the
modification affects multiple base tables.
```

6.4 **INSTEAD OF** Trigger — tornando visões complexas "atualizáveis"

Para visões com **JOIN** que precisam aceitar escrita, a solução é criar um **INSTEAD OF** trigger que intercepta a operação e a traduz manualmente para as tabelas base:

```
CREATE TRIGGER fiscal.trg_vw_notas_ativas_insert
ON fiscal.vw_notas_emitidas_ativas
INSTEAD OF INSERT
```

```

AS
BEGIN
    SET NOCOUNT ON;

    -- Redireciona o INSERT para a tabela base correta
    INSERT INTO fiscal.nota_fiscal (nrchaveacesso, dhemissao, vltotalnota, stnfe,
id_contribuinte_emitente)
    SELECT
        i.chave_acesso,
        i.data_emissao,
        i.valor_total,
        'A', -- situacao padrão para novas notas
        c.id_contribuinte
    FROM inserted AS i
    JOIN fiscal.contribuinte AS c ON c.cnpj = i.cnpj;
END;

```

7. WITH CHECK OPTION

WITH CHECK OPTION é uma cláusula que impede que operações de **INSERT** ou **UPDATE** através da visão produzam linhas que a própria visão **não enxergaria** — ou seja, garante que os dados inseridos/modificados via visão continuem visíveis por ela após a operação.

7.1 Problema sem **WITH CHECK OPTION**

```

CREATE VIEW fiscal.vw_contribuintes_simples_sem_check
AS
SELECT id_contribuinte, razao_social, cnpj, regime_tributario
FROM fiscal.contribuinte
WHERE regime_tributario = 'Simples Nacional';
-- Sem WITH CHECK OPTION

-- Este UPDATE muda o regime para 'Lucro Real' – a linha SAIRÁ da visão após o
update
-- mas o T-SQL permite sem reclamar:
UPDATE fiscal.vw_contribuintes_simples_sem_check
SET regime_tributario = 'Lucro Real'
WHERE cnpj = '12345678000195';
-- A linha agora não aparece mais na visão – "desapareceu silenciosamente"

```

7.2 Solução com **WITH CHECK OPTION**

```
CREATE VIEW fiscal.vw_contribuintes_simples
AS
SELECT id_contribuinte, razao_social, cnpj, regime_tributario
FROM fiscal.contribuinte
WHERE regime_tributario = 'Simples Nacional'
WITH CHECK OPTION; -- garante que dados modificados via visão permaneçam visíveis
por ela

-- Agora o mesmo UPDATE é bloqueado com mensagem de erro:
UPDATE fiscal.vw_contribuintes_simples
SET regime_tributario = 'Lucro Real'
WHERE cnpj = '12345678000195';
```

The attempted insert or update failed because the target view either specifies WITH CHECK OPTION or spans a view that specifies WITH CHECK OPTION and one or more rows resulting from the operation did not qualify under the CHECK OPTION constraint.

Regra prática fiscal: sempre use **WITH CHECK OPTION** em visões particionadas por critério de negócio (regime, município, circunscrição) que são expostas para escrita — isso evita que um analista "migre" inadvertidamente um contribuinte para fora de sua alçada de visibilidade.

8. WITH SCHEMABINDING

WITH SCHEMABINDING vincula a visão ao esquema das tabelas base, **impedindo que as tabelas referenciadas sejam alteradas** (colunas removidas, tabelas excluídas) enquanto a visão existir.

```
CREATE VIEW fiscal.vw_notas_ativas_schemabind
WITH SCHEMABINDING -- vincula ao schema atual das tabelas base
AS
SELECT
    n.nrchaveacesso AS chave_acesso,
    n.dhemissao AS data_emissao,
    n.vltotalnota AS valor_total,
    n.stnfe AS situacao,
    c.razao_social,
    c.cnpj
FROM fiscal.nota_fiscal AS n -- nome com schema qualificado: obrigatório
com SCHEMABINDING
JOIN fiscal.contribuinte AS c
```

```
ON c.id_contribuinte = n.id_contribuinte_emitente  
WHERE n.stnfe <> 'C';
```

Efeito prático:

```
-- Tentativa de remover coluna usada pela visão – será bloqueada:  
ALTER TABLE fiscal.nota_fiscal DROP COLUMN vltotalnota;  
-- Erro: Cannot DROP COLUMN 'vltotalnota' because it is referenced by object  
'vw_notas_ativas_schemabind'.
```

Regras obrigatórias com **WITH SCHEMABINDING**:

- Todos os objetos referenciados devem usar nomes **qualificados com schema** (`fiscal.nota_fiscal`, não apenas `nota_fiscal`).
- Não é permitido usar `SELECT *` — todas as colunas devem ser nomeadas explicitamente.
- **WITH SCHEMABINDING** é **pré-requisito obrigatório** para criar índices em visões (ver [seção 9](#)).

9. Visões Indexadas (Materialized Views)

Uma **visão indexada** (*indexed view*, também chamada de *materialized view*) é uma visão com **WITH SCHEMABINDING** sobre a qual é criado um **índice clusterizado** — fazendo com que o resultado da visão seja **armazenado fisicamente** no disco, como uma tabela real.

Isso é especialmente útil para relatórios fiscais pesados que precisam agregar milhões de notas fiscais repetidamente.

9.1 Pré-requisitos

- A visão deve ser criada com **WITH SCHEMABINDING**.
- O primeiro índice criado na visão deve ser **clusterizado e único** (**UNIQUE CLUSTERED**).
- A definição da visão não pode conter: subconsultas, **OUTER JOIN**, **DISTINCT**, **TOP**, CTEs, funções não determinísticas (`GETDATE()`, `NEWID()`), **UNION/EXCEPT/INTERSECT**, colunas calculadas não determinísticas.

- **SET ANSI_NULLS ON** e **SET QUOTED_IDENTIFIER ON** devem estar ativos na sessão.

9.2 Criando uma visão indexada fiscal

```
-- Passo 1: criar a visão com SCHEMABINDING e somente agregações suportadas
CREATE VIEW fiscal.vw_idx_totais_por_regime
WITH SCHEMABINDING
AS
SELECT
    c.regime_tributario,
    -- COUNT_BIG(*) é obrigatório quando há GROUP BY em visão indexada
    COUNT_BIG(*)                AS qtd_notas,
    SUM(n.vltotalnota)           AS total_emitido,
    SUM(n.vlicms)                AS total_icms
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
    ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C'
GROUP BY c.regime_tributario;

-- Passo 2: criar o índice clusterizado único (materializa os dados)
CREATE UNIQUE CLUSTERED INDEX idx_vw_totais_por_regime
    ON fiscal.vw_idx_totais_por_regime (regime_tributario);

-- Passo 3 (opcional): índice não clusterizado para outras colunas de filtro
CREATE NONCLUSTERED INDEX idx_vw_totais_total_emitido
    ON fiscal.vw_idx_totais_por_regime (total_emitido DESC);
```

9.3 Consultando a visão indexada

```
-- SQL Server Enterprise/Developer: pode usar a visão indexada automaticamente
-- mesmo em consultas que NÃO referenciam a visão diretamente (query rewrite)
SELECT regime_tributario, SUM(vltotalnota)
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C'
GROUP BY c.regime_tributario;

-- Para forçar o uso da visão indexada explicitamente (qualquer edição):
SELECT regime_tributario, total_emitido, qtd_notas
FROM fiscal.vw_idx_totais_por_regime WITH (NOEXPAND)
ORDER BY total_emitido DESC;
```

WITH (NOEXPAND): na edição Standard do SQL Server, o otimizador não usa visões indexadas automaticamente. A hint **NOEXPAND** força a leitura dos dados

materializados, evitando que a visão seja "expandida" de volta às tabelas base.

9.4 Manutenção automática

O SQL Server **mantém automaticamente** os dados da visão indexada sempre que as tabelas base são modificadas (INSERT/UPDATE/DELETE) — similar a um índice comum. Há custo extra nas escritas; o benefício compensa em cenários com muitas leituras e poucas escritas, típico de consolidações fiscais noturnas.

10. INFORMATION_SCHEMA e Metadados de Visões

O SQL Server expõe metadados de visões em diversas visões de sistema (*system views*) — que são, elas próprias, visões.

10.1 Listar todas as visões do schema fiscal

```
SELECT
    TABLE_SCHEMA      AS schema_nome,
    TABLE_NAME        AS visao_nome,
    VIEW_DEFINITION     AS definicao_sql
FROM INFORMATION_SCHEMA.VIEWS
WHERE TABLE_SCHEMA = 'fiscal'
ORDER BY TABLE_NAME;
```

10.2 Consultar via **sys.views** (mais completo)

```
SELECT
    v.name                                AS visao_nome,
    SCHEMA_NAME(v.schema_id)             AS schema_nome,
    v.create_date                         AS criada_em,
    v.modify_date                         AS alterada_em,
    v.is_schema_published,
    -- verifica se tem SCHEMABINDING
    OBJECTPROPERTY(v.object_id, 'IsSchemaBound') AS tem_schemabinding
FROM sys.views AS v
```



```
WHERE SCHEMA_NAME(v.schema_id) = 'fiscal'  
ORDER BY v.name;
```

10.3 Ver a definição de uma visão específica

```
-- Opção 1: via função de sistema  
SELECT OBJECT_DEFINITION(OBJECT_ID('fiscal.vw_notas_emitidas_ativas'));  
  
-- Opção 2: via sp_helptext (formata em múltiplas linhas)  
EXEC sp_helptext 'fiscal.vw_notas_emitidas_ativas';
```

10.4 Identificar dependências de uma visão

```
-- Quais tabelas e colunas a visão referencia?  
SELECT  
    referenced_schema_name AS tabela_schema,  
    referenced_entity_name AS tabela_nome,  
    referenced_minor_name AS coluna_nome  
FROM sys.dm_sql_referenced_entities('fiscal.vw_notas_emitidas_ativas', 'OBJECT')  
ORDER BY referenced_entity_name, referenced_minor_name;  
  
-- Quais objetos dependem desta tabela (impacto antes de alterar)?  
SELECT  
    referencing_schema_name AS objeto_schema,  
    referencing_entity_name AS objeto_nome,  
    referencing_class_desc AS tipo_objeto  
FROM sys.dm_sql_referencing_entities('fiscal.nota_fiscal', 'OBJECT')  
ORDER BY referencing_entity_name;
```

11. Visões x CTEs x Tabelas Temporárias: Quando Usar Cada Uma

Critério	Visão (VIEW)	CTE (WITH ... AS)	Tabela Temporária (#temp)
Escopo	Objeto permanente do banco de dados — existe entre sessões	Existe apenas dentro da instrução SQL que a define	Existe durante a sessão (ou conexão) corrente
Reutilização	Qualquer consulta, qualquer usuário autorizado	Apenas dentro da instrução onde foi declarada	Qualquer instrução na mesma sessão, após a criação
Armazenamento de dados	Nenhum (virtual) — exceto visão indexada	Nenhum (virtual)	Sim — armazena cópia física em tempdb
Segurança / controle de acesso	Ideal — permissão granular por objeto	Não — a permissão necessária é a da consulta subjacente	Não
Recursividade	Não suporta	Suporta (WITH RECURSIVE)	Não diretamente
Desempenho em grandes volumes	Depende do otimizador (pode expandir)	Depende do otimizador	Pode ter vantagem quando o resultado intermediário é reutilizado muitas vezes — o SQL Server materializa
Índice no resultado intermediário	Sim (visão indexada)	Não	Sim — pode criar índice na #temp
Caso de uso típico na SEFAZ-PB	Relatórios padronizados, controle de acesso por perfil, simplificação de	Estruturar consultas analíticas complexas dentro de um script	ETL e processamentos em lote — quando o resultado intermediário precisa de índice para ser consultado repetidamente

Critério	Visão (VIEW)	CTE (WITH ... AS)	Tabela Temporária (#temp)
	consultas recorrentes		

Regra prática:

Pergunta de negócio recorrente, multi-usuário, com controle de acesso → VIEW
 Consulta complexa de uso único, dentro de um script → CTE
 Processamento em lote, resultado intermediário grande consultado N vezes → #TEMP

12. Desempenho e Plano de Execução

Visões são **transparentes** ao otimizador de consultas: quando o SQL Server executa `SELECT * FROM fiscal.vw_notas_emitidas_ativas WHERE valor_total > 10000`, ele substitui internamente a visão pela sua definição e otimiza a consulta resultante como um todo — um processo chamado **view expansion** (expansão de visão).

Consequências práticas:

- Um filtro **WHERE** aplicado na consulta **sobre** a visão é propagado para dentro da visão pelo otimizador (*predicate pushdown*) — o SQL Server não precisa primeiro materializar todos os dados da visão para depois filtrar.
- Verificar o plano de execução real (**Ctrl+M**) confirma se o *predicate pushdown* ocorreu: o operador de leitura de tabela (*Index Seek* ou *Table Scan*) deve mostrar o filtro aplicado sobre a tabela base, não sobre a visão.

12.1 Situações que podem degradar desempenho com visões

```
-- Exemplo problemático: função não determinística impede pushdown eficiente
CREATE VIEW fiscal.vw_notas_recents
AS
SELECT n.nrchaveacesso, n.dhmissao, n.vltotalnota, c.razao_social
FROM fiscal.nota_fiscal AS n
```

```
JOIN fiscal.contribuinte AS c ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.dhemissao >= DATEADD(DAY, -30, GETDATE()); -- range dinâmico
```

Neste caso, cada consulta recalcula a janela de 30 dias — o índice em **dhemissao** ainda é usado (*range seek*), mas o intervalo muda a cada execução. Dependendo do volume, pode ser mais eficiente passar a data como parâmetro em uma **função com valor de tabela** (TVF) em vez de uma visão.

12.2 Dicas de desempenho para visões

- Garanta índices nas colunas de **JOIN** e **WHERE** das tabelas base referenciadas pela visão.
- Use **WITH SCHEMABINDING** + índice clusterizado (visão indexada) para relatórios de consolidação com alto volume de leituras.
- Evite visões que referenciam outras visões com mais de dois níveis de aninhamento — o otimizador pode ter dificuldade em gerar um plano eficiente.
- Para relatórios fiscais mensais pesados, considere **visões indexadas** atualizadas por job noturno, ou **tabelas de resumo** populadas por procedures agendadas.

13. Cenário Integrado: Painel de Auditoria Fiscal

O conjunto de visões a seguir forma a camada de acesso do **Painel de Auditoria Fiscal da SEFAZ-PB**, combinando diversas técnicas deste módulo.

13.1 Visão base: notas ativas com dados do contribuinte

```
CREATE VIEW fiscal.vw_notas_ativas
WITH SCHEMABINDING
AS
SELECT
    n.nrchaveacesso      AS chave_acesso,
    n.dhemissao          AS data_emissao,
    n.vltotalnota        AS valor_total,
    n.vlicms              AS valor_icms,
```

```

n.vlbasecalculado      AS base_calculo_icms,
n.stnfe                AS situacao,
n.cdmodelo             AS modelo_nf,
c.id_contribuinte,
c.razao_social,
c.cnpj,
c.regime_tributario,
c.municipio,
c.id_circunscricao
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
  ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C';

```

13.2 Visão de divergências de ICMS

Identifica notas onde o ICMS declarado diverge do esperado pela alíquota do regime:

```

CREATE OR ALTER VIEW fiscal.vw_divergencias_icms
AS
SELECT
  na.chave_acesso,
  na.data_emissao,
  na.razao_social,
  na.cnpj,
  na.regime_tributario,
  na.valor_total,
  na.base_calculo_icms,
  na.valor_icms                                AS icms_declarado,
  -- alíquota esperada por regime (tabela de parâmetros fiscais)
  pf.aliquota_icms                             AS aliquota_esperada,
  ROUND(na.base_calculo_icms * pf.aliquota_icms / 100, 2) AS icms_esperado,
  -- diferença absoluta
  na.valor_icms
    - ROUND(na.base_calculo_icms * pf.aliquota_icms / 100, 2) AS diferenca_icms
FROM fiscal.vw_notas_ativas AS na
JOIN fiscal.parametro_fiscal AS pf
  ON pf.regime_tributario = na.regime_tributario
-- filtra apenas notas com diferença superior a R$ 10,00 (tolerância operacional)
WHERE ABS(
  na.valor_icms
    - ROUND(na.base_calculo_icms * pf.aliquota_icms / 100, 2)
) > 10.00;

```

13.3 Visão de contribuintes omissos

Contribuintes que não emitiram nenhuma nota nos últimos 90 dias:

```

CREATE OR ALTER VIEW fiscal.vw_contribuintes_omissos
AS
SELECT
    c.id_contribuinte,
    c.razao_social,
    c.cnpj,
    c.regime_tributario,
    c.municipio,
    c.situacao_cadastral,
    -- data da última nota emitida (NULL se nunca emitiu)
    (
        SELECT MAX(n.dhmissao)
        FROM fiscal.nota_fiscal AS n
        WHERE n.id_contribuinte_emitente = c.id_contribuinte
              AND n.stnfe <> 'C'
    ) AS ultima_emissao
FROM fiscal.contribuinte AS c
WHERE c.situacao_cadastral = 'REGULAR'
      AND NOT EXISTS (
        SELECT 1
        FROM fiscal.nota_fiscal AS n
        WHERE n.id_contribuinte_emitente = c.id_contribuinte
              AND n.stnfe <> 'C'
              AND n.dhmissao >= DATEADD(DAY, -90, GETDATE())
      );

```

13.4 Visão protegida por perfil de circunscrição

```

CREATE OR ALTER VIEW fiscal.vw_divergencias_minha_area
AS
SELECT d.*
FROM fiscal.vw_divergencias_icms AS d
JOIN fiscal.contribuinte AS c
    ON c.cnpj = d.cnpj
WHERE c.id_circunscricao = (
    SELECT af.id_circunscricao
    FROM fiscal.analista_fiscal AS af
    WHERE af.login_windows = SUSER_SNAME()
);

```

13.5 Visão indexada para consolidação gerencial

```

CREATE VIEW fiscal.vw_idx_consolidado_mensal
WITH SCHEMABINDING
AS
SELECT
    c.regime_tributario,
    YEAR(n.dhmissao) AS ano_emissao,
    MONTH(n.dhmissao) AS mes_emissao,
    COUNT_BIG(*) AS qtd_notas,
    SUM(n.vltotalnota) AS total_emitido,
    SUM(n.vlicms) AS total_icms
FROM fiscal.nota_fiscal AS n
JOIN fiscal.contribuinte AS c
    ON c.id_contribuinte = n.id_contribuinte_emitente
WHERE n.stnfe <> 'C'
GROUP BY
    c.regime_tributario,
    YEAR(n.dhmissao),
    MONTH(n.dhmissao);

CREATE UNIQUE CLUSTERED INDEX idx_consolidado_mensal
ON fiscal.vw_idx_consolidado_mensal
(regime_tributario, ano_emissao, mes_emissao);

```

Consultando o painel consolidado (sem percorrer milhões de notas):

```

SELECT
    regime_tributario,
    ano_emissao,
    mes_emissao,
    qtd_notas,
    total_emitido,
    total_icms,
    ROUND(total_icms * 100.0 / NULLIF(total_emitido, 0), 2) AS perc_icms
FROM fiscal.vw_idx_consolidado_mensal WITH (NOEXPAND)
WHERE ano_emissao = 2026
ORDER BY regime_tributario, mes_emissao;

```

14. Boas Práticas

- **Nomeie visões com prefixo `vw_`** e nomes descritivos que comuniquem a intenção (`vw_contribuintes_omissos`, `vw_divergencias_icms`) — evite nomes genéricos como `vw1` ou `vw_dados`.
- **Prefira `CREATE OR ALTER VIEW` a `DROP + CREATE`** — preserva permissões e evita janelas de indisponibilidade.

- **Jamais use `SELECT *`** na definição de uma visão — adicionar colunas na tabela base não as propaga automaticamente para a visão, mas pode causar comportamento inesperado se o `SELECT *` for depois usado em `INSERT ... SELECT`.
- **Qualifique sempre o schema** nos nomes de objetos dentro da visão (`fiscal.nota_fiscal`, não apenas `nota_fiscal`) — obrigatório para `WITH SCHEMABINDING` e boa prática em geral.
- **Use `WITH CHECK OPTION`** em visões particionadas por critério de negócio que permitem escrita — garante que um `UPDATE` não mova inadvertidamente um registro para fora do escopo da visão.
- **Use `WITH SCHEMABINDING`** em visões que fazem parte da camada de relatórios críticos — protege contra alterações acidentais nas tabelas base.
- **Documente a intenção de negócio** da visão em um comentário antes do `CREATE VIEW` ou em um catálogo de objetos — o código SQL não captura "por que esta visão existe", apenas "o que ela faz".
- **Evite visões aninhadas com mais de dois níveis** — `vw_a` referenciando `vw_b` que referencia `vw_c` dificulta o entendimento do plano de execução e a manutenção.
- **Avalie visões indexadas** para agregações que são consultadas com frequência sobre grandes volumes — o custo extra nas escritas geralmente é compensado em ambientes OLAP/relatórios.
- **Valide o plano de execução (`Ctrl+M`)** após criar uma visão usada em consultas críticas — confirme que o otimizador está fazendo *predicate pushdown* e usando os índices esperados.

15. Glossário

Termo	Definição
Visão (View)	Objeto de banco de dados que armazena uma instrução <code>SELECT</code> nomeada, funcionando como tabela virtual
Tabela base	Tabela física que fornece os dados para a visão
View expansion	Processo pelo qual o otimizador substitui a referência à visão pela sua definição antes de gerar o plano de execução

Termo	Definição
Predicate pushdown	Técnica do otimizador que propaga filtros da consulta externa para dentro da visão, evitando materialização desnecessária
Visão indexada (Indexed View)	Visão com WITH SCHEMABINDING que possui um índice clusterizado, fazendo com que seu resultado seja armazenado fisicamente
WITH SCHEMABINDING	Cláusula que vincula a visão ao schema das tabelas base, impedindo alterações estruturais que quebrariam a visão
WITH CHECK OPTION	Cláusula que garante que INSERT/UPDATE via visão nunca produzam linhas fora do filtro da própria visão
INSTEAD OF Trigger	Trigger que intercepta operações DML em uma visão e as redireciona manualmente para as tabelas base
NOEXPAND	Hint de tabela que força o uso dos dados materializados de uma visão indexada, sem expandir a definição da visão
Row-Level Security (RLS)	Recurso nativo do SQL Server 2016+ para filtrar linhas por usuário, alternativa mais robusta ao filtro por SUSER_SNAME() em visões

16. Exercícios

Parte I — Criação e manipulação básica

1. Crie uma visão **fiscal.vw_nfce_ativas** que exiba apenas as NFC-e (modelo 65) com situação diferente de cancelada. Inclua: **chave_acesso**, **data_emissao**, **valor_total**, **razao_social**, **cnpj** e **municipio**.
2. Altere a visão criada no exercício anterior para incluir a coluna **regime_tributario** sem perder as permissões concedidas. Use a instrução correta para isso.
3. Liste todos os objetos do schema **fiscal** do tipo visão, exibindo nome, data de criação e se possuem **SCHEMABINDING**. Use **sys.views**.
4. Exiba a definição (código SQL) da visão criada no exercício 1 usando **OBJECT_DEFINITION**.

Parte II — Segurança e controle de acesso

5. Crie uma visão `fiscal.vw_nfe_publica` que exiba apenas `chave_acesso`, `data_emissao`, `valor_total`, `municipio` e `regime_tributario`, **maskando o CNPJ** (exibindo somente os 8 primeiros dígitos). Essa visão deve ser usada para compartilhamento público de dados agregados.
6. Escreva os comandos `GRANT/DENY` para conceder acesso à `vw_nfe_publica` ao perfil `[perfil_portal_transparencia]` e negar acesso direto à tabela `fiscal.nota_fiscal` para o mesmo perfil.
7. Crie uma visão `fiscal.vw_notas_por_municipio` que filtre automaticamente as notas pelo município do analista logado, usando `SUSER_SNAME()` e uma tabela fictícia `fiscal.analista_municipal` (`login_windows`, `municipio`).

Parte III — `WITH CHECK OPTION` e `WITH SCHEMABINDING`

8. Crie uma visão `fiscal.vw_contribuintes_mei` para contribuintes do regime "MEI" e tente realizar um `UPDATE` que mude o regime para "Simples Nacional". Observe o comportamento **sem** e **com** `WITH CHECK OPTION`.
9. Crie a visão do exercício 8 com `WITH SCHEMABINDING`. Em seguida, tente executar `ALTER TABLE fiscal.contribuinte DROP COLUMN regime_tributario` e observe o erro gerado. Explique por que isso é útil em produção.

Parte IV — Visões indexadas e desempenho

10. Crie uma visão indexada `fiscal.vw_idx_totais_municipio` que agregue, por município e mês, a quantidade de notas e o total emitido. Crie o índice clusterizado único. Consulte-a com `WITH (NOEXPAND)`.
11. Compare o plano de execução (`Ctrl+M`) de uma consulta de agregação mensal executada:
 - (a) Diretamente nas tabelas base com `GROUP BY`.
 - (b) Usando a visão indexada do exercício 10 com `WITH (NOEXPAND)`.Identifique em qual operador está a maior diferença de custo estimado.
12. Crie uma visão `fiscal.vw_omissas_simples` para contribuintes do Simples Nacional sem emissão de NF nos últimos 60 dias. Use `NOT EXISTS` para identificar a ausência de notas.

Desafio

Projete e implemente um **conjunto de visões** para o Painel de Auditoria de Risco Fiscal da SEFAZ-PB, atendendo aos seguintes requisitos:

Visão 1 — `fiscal.vw_risco_alto`: contribuintes com pelo menos uma nota de valor acima de R\$ 500.000 no trimestre vigente E com histórico de cancelamento acima de 10% das notas emitidas no mesmo período.

Visão 2 — `fiscal.vw_top10_emissores`: os 10 maiores emissores de cada regime tributário (por valor total emitido no mês corrente), usando subconsulta correlacionada ou `CROSS APPLY` na definição da visão.

Visão 3 — `fiscal.vw_painel_gerencial` (indexada): consolidação mensal por regime e UF de destino, com totais de notas, valor emitido, ICMS e percentual de ICMS sobre o valor emitido. Inclua índice clusterizado.

Requisitos gerais:

- Todas as visões com `WITH SCHEMABINDING` onde aplicável.
- `WITH CHECK OPTION` nas visões que permitem escrita.
- Script de `GRANT` para três perfis: `[perfil_auditor_senior]`, `[perfil_analista_operacional]` e `[perfil_gestor_tributario]` — com permissões distintas para cada visão.
- Exiba o plano de execução da consulta sobre a visão indexada e identifique se o SQL Server usou os dados materializados.

17. Referências

- **Microsoft Learn — CREATE VIEW (Transact-SQL):** referência oficial de sintaxe, restrições e opções de visões no SQL Server (learn.microsoft.com/sql/t-sql/statements/create-view-transact-sql).
- **Microsoft Learn — Indexed Views:** documentação completa sobre visões indexadas, pré-requisitos e limitações.
- **Microsoft Learn — Row-Level Security:** recurso nativo do SQL Server 2016+ como alternativa ao filtro por `SUSER_SNAME()` em visões de segurança.
- **Microsoft Learn — sys.views (Transact-SQL):** catálogo de visões de sistema para inspeção de metadados.

- **Microsoft Learn — `sys.dm_sql_referenced_entities`:** DMF para análise de dependências de objetos.
- **Itzik Ben-Gan — "T-SQL Fundamentals"** (Microsoft Press): capítulo sobre visões, discutindo atualizabilidade, **INSTEAD OF** triggers e visões indexadas.
- Consulte também [subconsultas.md](#) para revisão de CTEs e subconsultas correlacionadas utilizadas na definição de visões complexas.